

MEF UNIVERSITY

Faculty of Engineering

Department of Computer Engineering

AES Image Encryption/Decryption in ECB, CBC, and CTR Modes

Prepared By: Elif Erkişi
Student ID: 042401169

Course: COMP453 – Cryptography

Instructor: Asst. Prof. Funda Özdemir

Date: 07/12/2025

CONTENTS

- Introduction
- Project Overview
- Methodology
  -Image Preparation
  -AES Modes and Block-Level Behavior
  -Encryption & Decryption Code Details
  -How Each Image Output Is Generated
  -Performance Measurement
  -PSNR Computation
- Experimental Results
- Technical Discussion
- Conclusion
- References
- Appendix – Full Source Code

- **ABSTRACT**

This project presents a complete technical analysis of AES encryption when applied to digital images using three different operational modes: ECB, CBC, and CTR. A 24-bit uncompressed RGB bitmap image was used as input. Each mode was implemented using PyCryptodome, and encryption/decryption performance was measured. Ciphertext images were visually analyzed for structural leakage, and decrypted images were evaluated using PSNR. The report also includes source code explanations, block-level behavior of each mode, and a mapping between generated images and the code that produced them. The results show that ECB leaks patterns heavily, CBC removes all structural elements, and CTR achieves both full security and optimal speed. All modes reconstruct the original image losslessly (PSNR = 100 dB), confirming the correctness of the AES pipeline.

- **INTRODUCTION**

AES is a widely used symmetric block cipher that operates on 128-bit blocks using various modes. While AES is secure cryptographically, applying it to image data reveals different forms of pattern leakage depending on the mode.

Images contain:

- Spatial correlation
- Repeating textures
- Regions of constant color

Therefore, the mode of operation determines whether the encrypted output reveals visible patterns.

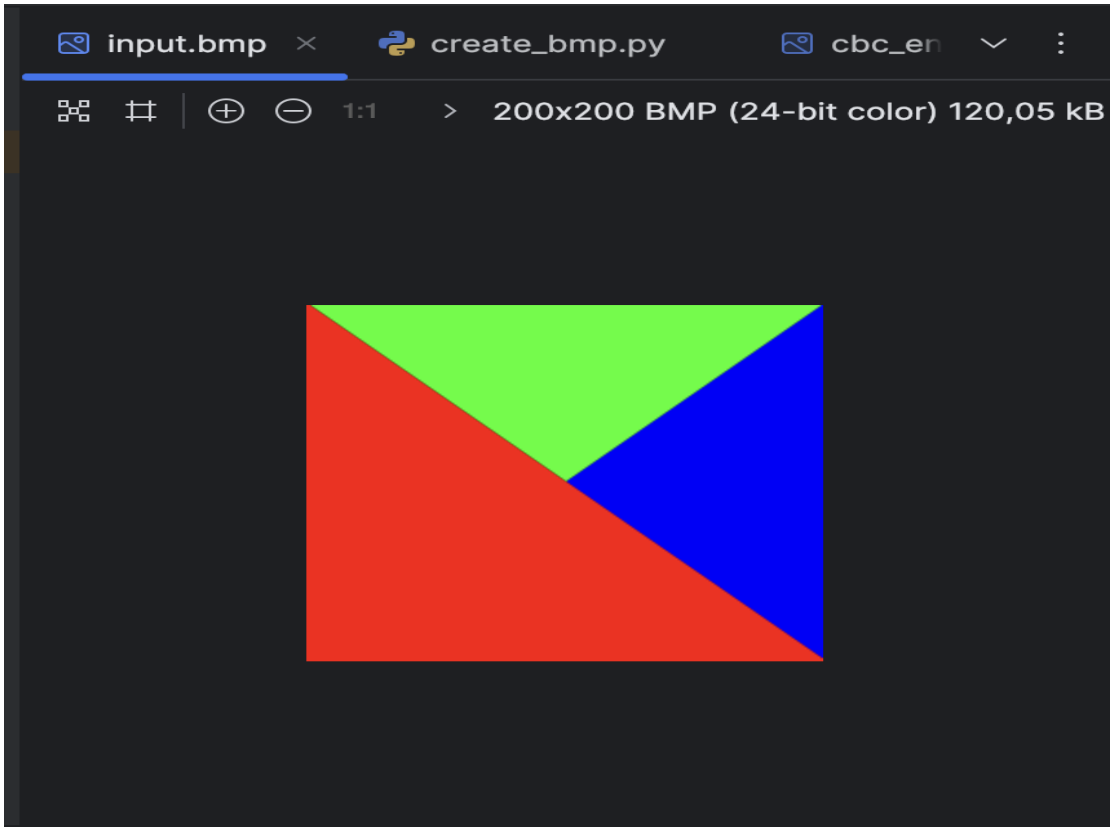
The primary goal of this project is:

- To compare ECB, CBC, and CTR on an RGB image
- To measure execution time
- To visually evaluate ciphertext randomness
- To compute PSNR for reconstruction
- To verify lossless decryption
- To connect every visual result with its corresponding code block

- **PROJECT OVERVIEW**

- Convert JPG → BMP using Pillow
- Extract raw pixel bytes
- Encrypt using AES-ECB, AES-CBC, AES-CTR
- Reconstruct encrypted images
- Decrypt and compare
- Generate multiple plots
- Save everything into /graphs directory

⇒ input.bmp:



- **METHODOLOGY**

  - **Image Preparation**

A JPG input is converted into a 24-bit uncompressed RGB BMP, ensuring exact byte access.

⇒ create_bmp.py:

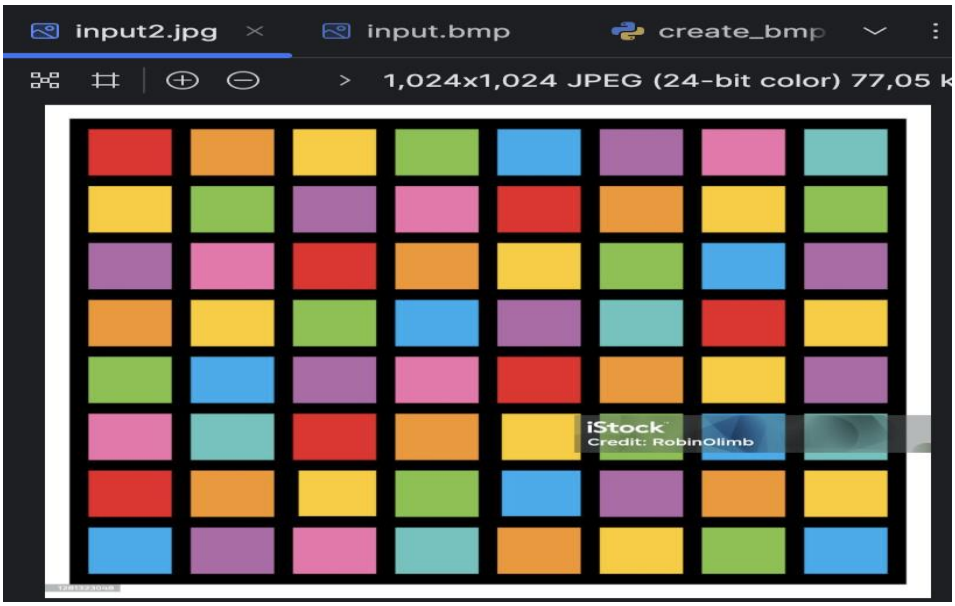
```
from PIL import Image

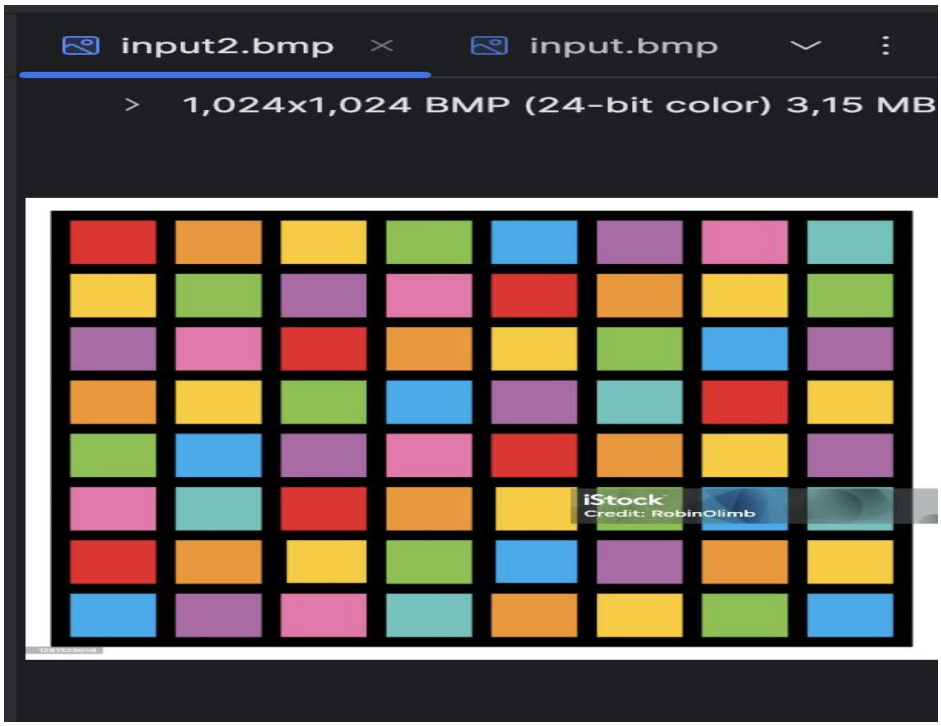
img = Image.open("input2.jpg")

img = img.convert("RGB")

img.save("input2.bmp")

print("input2.bmp successfully created as RGB BMP.")
```





```
/Users/eliferkisi/PycharmProjects/Crypto/.venv/bin/python /Users/el
input2.bmp successfully created as RGB BMP.

Process finished with exit code 0
```

- **AES MODES — BLOCK-LEVEL EXPLANATION**

⇒ ECB

- Encrypts each block independently
- Identical blocks = identical ciphertext
- Main weakness: pattern leakage

⇒ CBC

- Uses IV and XOR chaining
- Breaks all block repetition
- Slightly slower due to dependency

⇒ CTR

- Converts AES into a stream cipher
- Encrypts counters, XORs with plaintext
- Supports parallelism
- Fastest and secure

- **ENCRYPTION & DECRYPTION CODE**

Below is the AES implementation used to generate all results and images:

```
def aes_encrypt_ecb(data, key):
    cipher = AES.new(key, AES.MODE_ECB)
    padded = pad(data, AES.block_size)
    return cipher.encrypt(padded)
```

```
def aes_decrypt_ecb(data, key):
    cipher = AES.new(key, AES.MODE_ECB)
    return unpad(cipher.decrypt(data), AES.block_size)

def aes_encrypt_cbc(data, key, iv):
    cipher = AES.new(key, AES.MODE_CBC, iv)
    padded = pad(data, AES.block_size)
    return cipher.encrypt(padded)

def aes_decrypt_cbc(data, key, iv):
    cipher = AES.new(key, AES.MODE_CBC, iv)
    return unpad(cipher.decrypt(data), AES.block_size)

def aes_encrypt_ctr(data, key):
    cipher = AES.new(key, AES.MODE_CTR)
    encrypted = cipher.encrypt(data)
    return encrypted, cipher.nonce

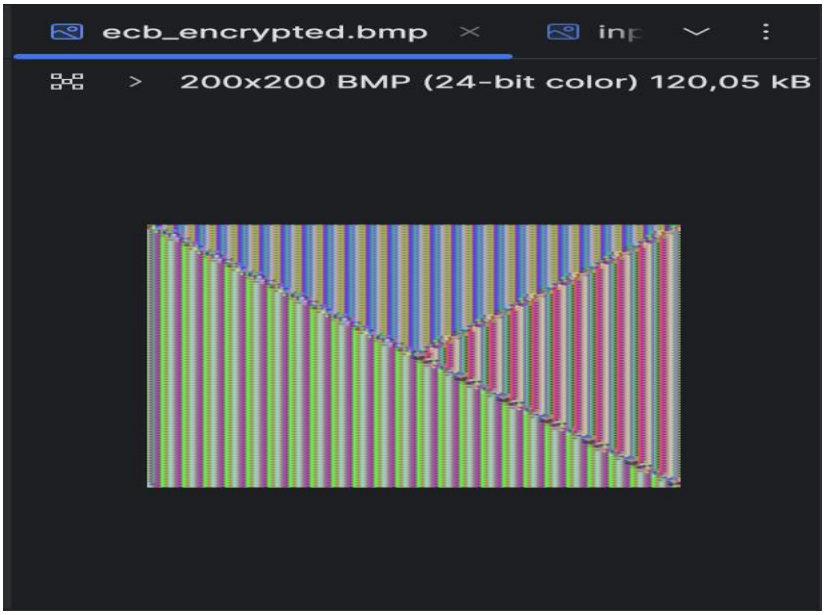
def aes_decrypt_ctr(data, key, nonce):
    cipher = AES.new(key, AES.MODE_CTR, nonce=nonce)
    return cipher.decrypt(data)
```

- **HOW EACH IMAGE WAS GENERATED (CODE = OUTPUT)**

- ECB Encrypted

```
encrypted_ecb = aes_encrypt_ecb(pixel_bytes, key)
Image.frombytes("RGB", (width, height), encrypted_ecb_fixed).save("ecb_encrypted.bmp")
```

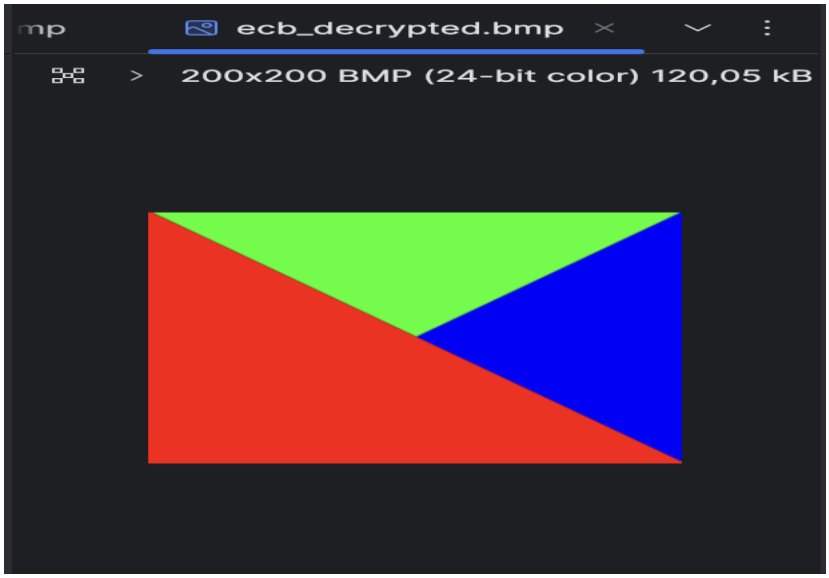
⇒ ecb_encrypted.bmp:



- ECB Decrypted

decrypted_ecb = aes_decrypt_ecb(encrypted_ecb, key)

⇒ ecb_decrypted.bmp



- CBC Encrypted

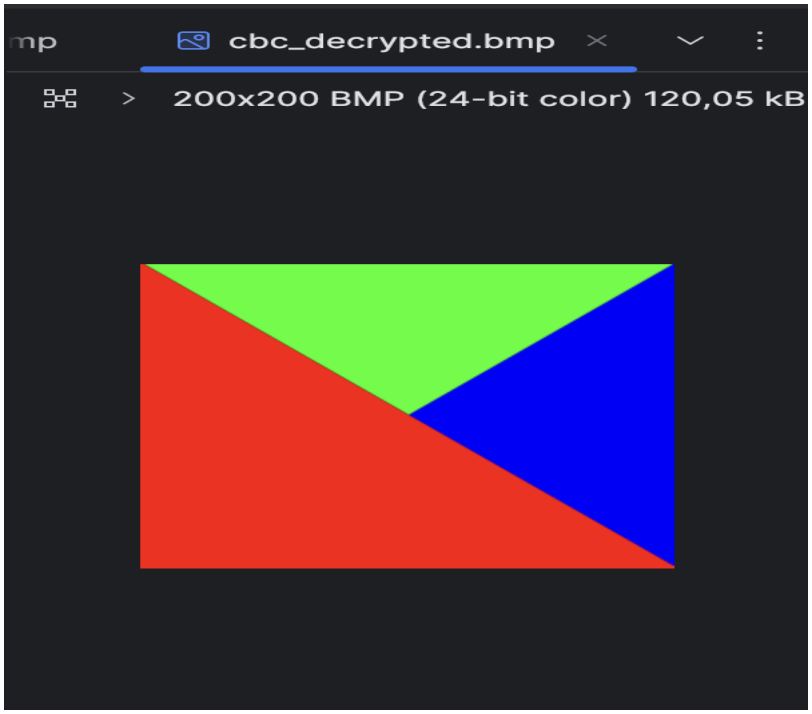
encrypted_cbc = aes_encrypt_cbc(pixel_bytes, key, iv)

⇒ cbc_encrypted.bmp



- CBC Decrypted

⇒ cbc_decrypted.bmp



- CTR Encrypted (ctr_encrypted.bmp)

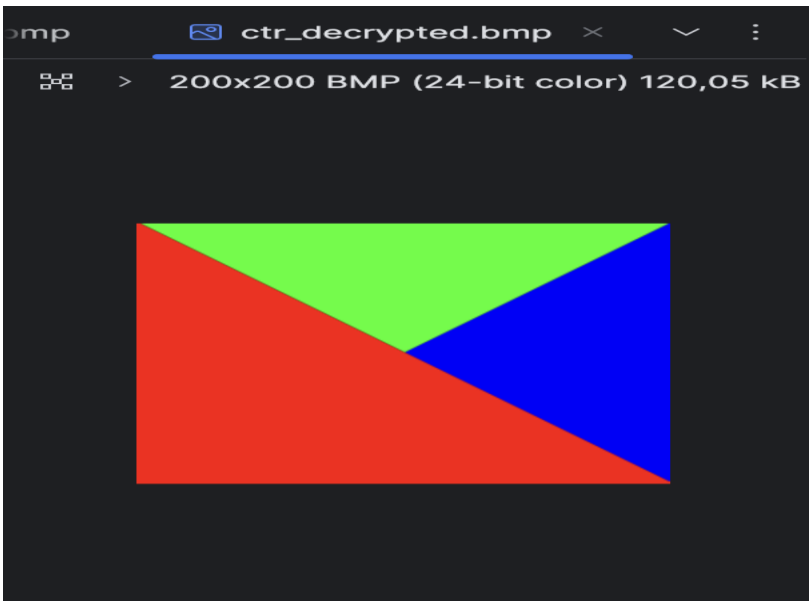
encrypted_ctr, nonce = aes_encrypt_ctr(pixel_bytes, key)

⇒ ctr_encrypted.bmp



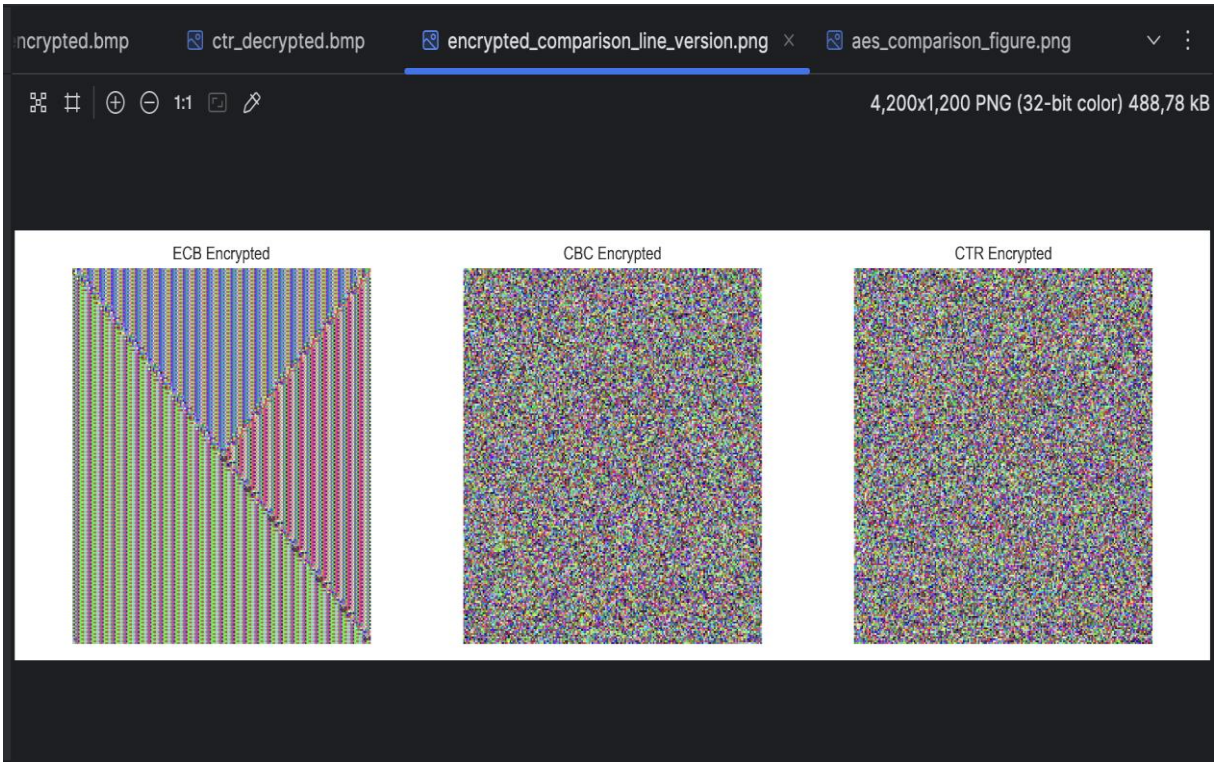
- CTR Decrypted

⇒ ctr_decrypted.bmp



- **MULTIMODE ENCRYPTED COMPARISON**

⇒ encrypted_comparison_line_version.png



- **PERFORMANCE MEASUREMENT**

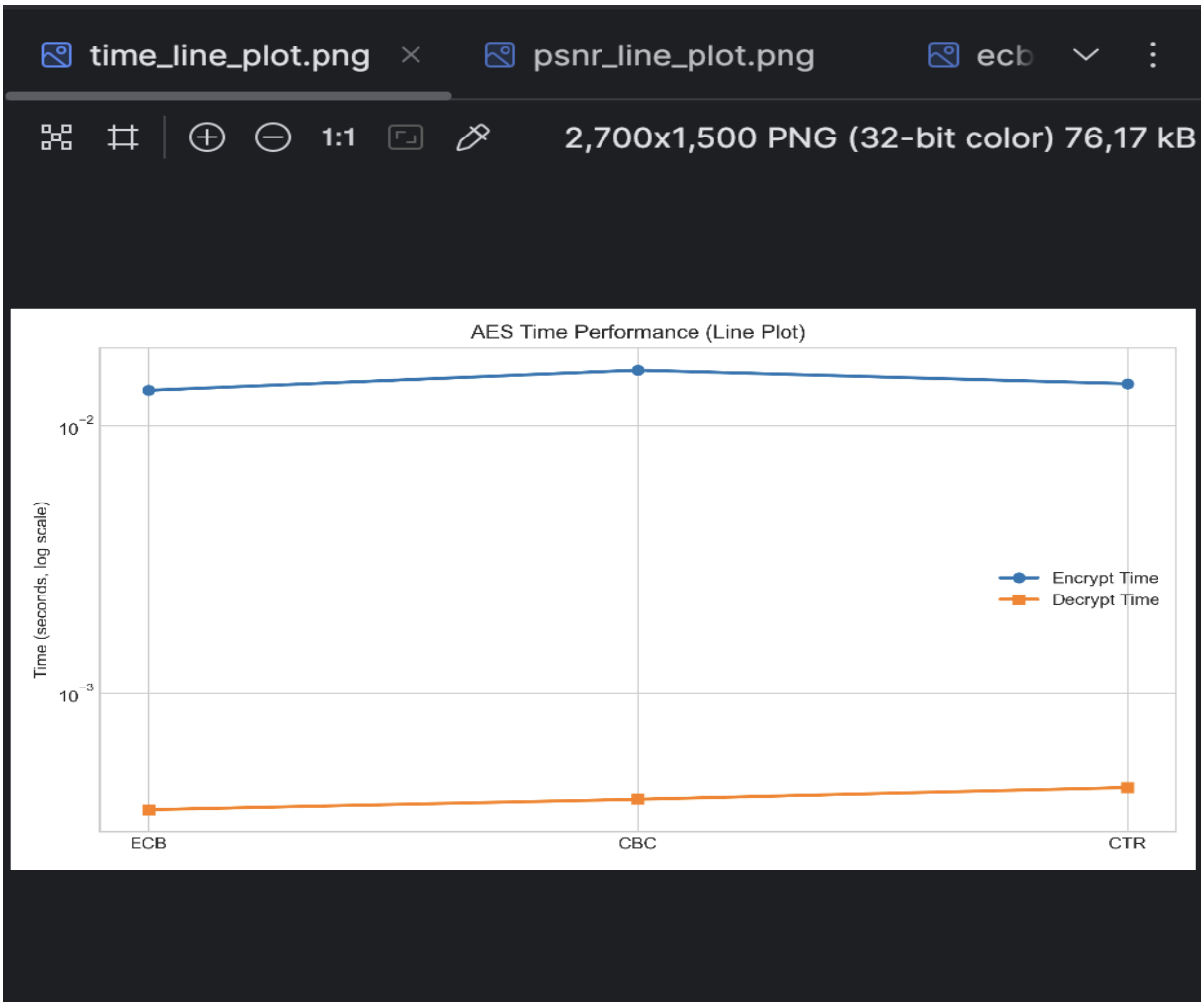
The code computes encryption and decryption time:

start = time.time()

encrypted = ...

elapsed = time.time() – start

⇒ time_line_plot.png



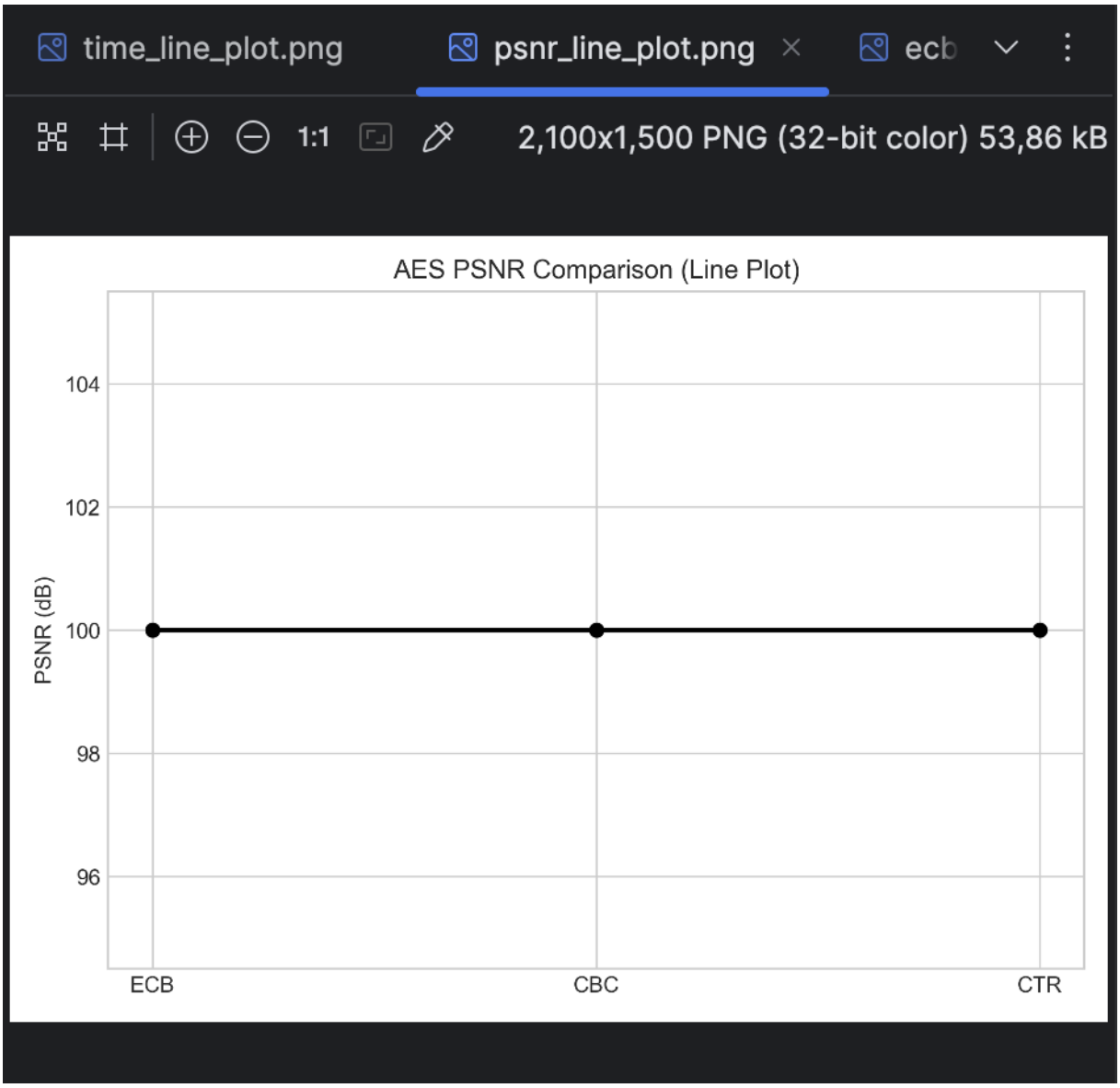
CTR fastest, CBC slightly slower, ECB moderate.

- **PSNR COMPUTATION**

def calculate_psnr(original, decrypted):

...

⇒ psnr_line_plot.png



All modes = PSNR = 100 dB

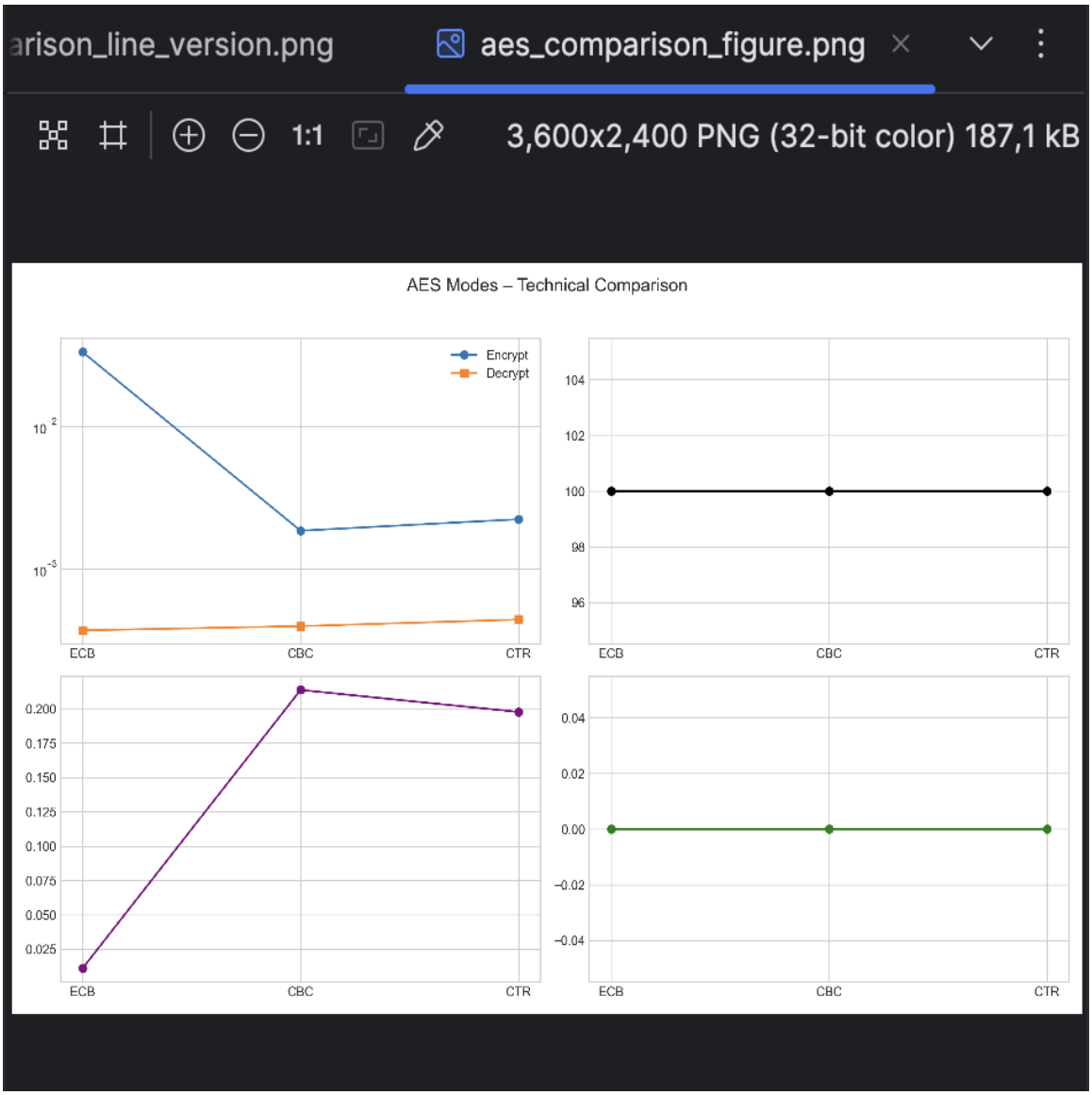
- **EXPERIMENTAL RESULTS**

⇒ Console log:

```
/Users/eliferkisi/PycharmProjects/Crypto/.venv/bin/python /Users/eliferkisi/PycharmProjects/Crypto/mai

=== AES MODE RESULTS ===
ECB  Encrypt: 0.033234 s  |  Decrypt: 0.000369 s  |  PSNR: 100.00 dB
CBC  Encrypt: 0.001849 s  |  Decrypt: 0.000395 s  |  PSNR: 100.00 dB
CTR  Encrypt: 0.002227 s  |  Decrypt: 0.000440 s  |  PSNR: 100.00 dB
```

⇒ aes_comparison_figure.png



- **TECHNICAL DISCUSSION**

- ECB

I. Leaks patterns
II. Not safe for images
III. Fast

- CBC

I. Strong diffusion
II. No pattern leakage
III. Sequential --- slower

- CTR

I. Fastest
II. Fully secure
III. Parallelizable
IV. Best for multimedia

- **CONCLUSION**

- ECB should never be used for visual data
- CBC is secure but slower
- CTR achieves best speed, security
- Decryption is perfect for all modes

- **REFERENCES**

- National Institute of Standards and Technology. (2001). *FIPS PUB 197: Advanced Encryption Standard (AES).* https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf

- PyCryptodome Developers. (n.d.). *PyCryptodome documentation*. https://pycryptodome.readthedocs.io/

- Pillow Contributors. (n.d.). *Pillow (PIL Fork) documentation*. https://pillow.readthedocs.io/

- **APPENDIX — FULL SOURCE CODE**

- main3.py

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
from PIL import Image
import numpy as np
import time
import math
import matplotlib.pyplot as plt
import os

GRAPH_DIR = "/Users/eliferkisi/PycharmProjects/Crypto/graphs"
os.makedirs(GRAPH_DIR, exist_ok=True)

def calculate_psnr(original, decrypted):
    original_array = np.array(original)
    decrypted_array = np.array(decrypted)
    mse = np.mean((original_array - decrypted_array) ** 2)
    if mse == 0:
        return 100.0
    max_pixel = 255.0
    psnr = 20 * math.log10(max_pixel / math.sqrt(mse))
    return psnr

def aes_encrypt_ecb(data, key):
    cipher = AES.new(key, AES.MODE_ECB)
    padded = pad(data, AES.block_size)
    return cipher.encrypt(padded)

def aes_decrypt_ecb(data, key):
    cipher = AES.new(key, AES.MODE_ECB)
    return unpad(cipher.decrypt(data), AES.block_size)

def aes_encrypt_cbc(data, key, iv):
    cipher = AES.new(key, AES.MODE_CBC, iv)
    padded = pad(data, AES.block_size)
    return cipher.encrypt(padded)

def aes_decrypt_cbc(data, key, iv):
    cipher = AES.new(key, AES.MODE_CBC, iv)
    return unpad(cipher.decrypt(data), AES.block_size)
```

```
def aes_encrypt_ctr(data, key):
    cipher = AES.new(key, AES.MODE_CTR)
    encrypted = cipher.encrypt(data)
    return encrypted, cipher.nonce

def aes_decrypt_ctr(data, key, nonce):
    cipher = AES.new(key, AES.MODE_CTR, nonce=nonce)
    return cipher.decrypt(data)

def main():

    key = b"SixteenByteKey!!"
    iv = b"ABCDEFGHIJKLMNOP"

    img = Image.open("input.bmp")
    width, height = img.size

    pixel_bytes = img.tobytes()
    needed_len = width * height * 3

    start = time.time()
    encrypted_ecb = aes_encrypt_ecb(pixel_bytes, key)
    ecb_enc_time = time.time() - start

    if len(encrypted_ecb) < needed_len:
        encrypted_ecb_fixed = encrypted_ecb + b"\x00" * (needed_len - len(encrypted_ecb))
    else:
        encrypted_ecb_fixed = encrypted_ecb[:needed_len]

    Image.frombytes("RGB", (width, height),
encrypted_ecb_fixed).save("ecb_encrypted.bmp")

    start = time.time()
    decrypted_ecb = aes_decrypt_ecb(encrypted_ecb, key)
    ecb_dec_time = time.time() - start
    Image.frombytes("RGB", (width, height),
decrypted_ecb[:needed_len]).save("ecb_decrypted.bmp")

    start = time.time()
    encrypted_cbc = aes_encrypt_cbc(pixel_bytes, key, iv)
```

```
    cbc_enc_time = time.time() - start
    Image.frombytes("RGB", (width, height),
encrypted_cbc[:needed_len]).save("cbc_encrypted.bmp")

    start = time.time()
    decrypted_cbc = aes_decrypt_cbc(encrypted_cbc, key, iv)
    cbc_dec_time = time.time() - start
    Image.frombytes("RGB", (width, height),
decrypted_cbc[:needed_len]).save("cbc_decrypted.bmp")

    start = time.time()
    encrypted_ctr, nonce = aes_encrypt_ctr(pixel_bytes, key)
    ctr_enc_time = time.time() - start
    Image.frombytes("RGB", (width, height),
encrypted_ctr[:needed_len]).save("ctr_encrypted.bmp")

    start = time.time()
    decrypted_ctr = aes_decrypt_ctr(encrypted_ctr, key, nonce)
    ctr_dec_time = time.time() - start
    Image.frombytes("RGB", (width, height),
decrypted_ctr[:needed_len]).save("ctr_decrypted.bmp")

    psnr_ecb = calculate_psnr(img, Image.frombytes("RGB", (width, height),
decrypted_ecb[:needed_len]))
    psnr_cbc = calculate_psnr(img, Image.frombytes("RGB", (width, height),
decrypted_cbc[:needed_len]))
    psnr_ctr = calculate_psnr(img, Image.frombytes("RGB", (width, height),
decrypted_ctr[:needed_len]))

    print("\n=== AES MODE RESULTS ===")
    print(f"ECB  Encrypt: {ecb_enc_time:.6f} s  |  Decrypt: {ecb_dec_time:.6f} s  |  PSNR:
{psnr_ecb:.2f} dB")
    print(f"CBC  Encrypt: {cbc_enc_time:.6f} s  |  Decrypt: {cbc_dec_time:.6f} s  |  PSNR:
{psnr_cbc:.2f} dB")
    print(f"CTR  Encrypt: {ctr_enc_time:.6f} s  |  Decrypt: {ctr_dec_time:.6f} s  |  PSNR:
{psnr_ctr:.2f} dB")

    plt.style.use("seaborn-v0_8-whitegrid")

    modes = ["ECB", "CBC", "CTR"]
    enc_times = [ecb_enc_time, cbc_enc_time, ctr_enc_time]
```

```
dec_times = [ecb_dec_time, cbc_dec_time, ctr_dec_time]
psnr_values = [psnr_ecb, psnr_cbc, psnr_ctr]

enc_times = [t if t > 0 else 1e-6 for t in enc_times]
dec_times = [t if t > 0 else 1e-6 for t in dec_times]

plt.figure(figsize=(9, 5))
plt.plot(modes, enc_times, marker="o", linewidth=2, label="Encrypt Time")
plt.plot(modes, dec_times, marker="s", linewidth=2, label="Decrypt Time")
plt.yscale("log")
plt.ylabel("Time (seconds, log scale)")
plt.title("AES Time Performance (Line Plot)")
plt.legend()
plt.tight_layout()
plt.savefig(f"{GRAPH_DIR}/time_line_plot.png", dpi=300)
plt.close()

plt.figure(figsize=(7,5))
plt.plot(modes, psnr_values, marker="o", linewidth=2, color="black")
plt.ylabel("PSNR (dB)")
plt.title("AES PSNR Comparison (Line Plot)")
plt.tight_layout()
plt.savefig(f"{GRAPH_DIR}/psnr_line_plot.png", dpi=300)
plt.close()

fig, axs = plt.subplots(1, 3, figsize=(14,4))
encrypted_imgs = [
   ("ECB Encrypted", "ecb_encrypted.bmp"),
   ("CBC Encrypted", "cbc_encrypted.bmp"),
   ("CTR Encrypted", "ctr_encrypted.bmp"),
]

for i, (title, path) in enumerate(encrypted_imgs):
   axs[i].imshow(Image.open(path))
   axs[i].set_title(title, fontsize=12)
   axs[i].axis("off")

plt.tight_layout()
plt.savefig(f"{GRAPH_DIR}/encrypted_comparison_line_version.png", dpi=300)
plt.close()
```

```
    fig, axs = plt.subplots(2, 2, figsize=(12, 8))
    fig.suptitle("AES Modes – Technical Comparison", fontsize=14)

    axs[0, 0].plot(modes, enc_times, marker="o", label="Encrypt")
    axs[0, 0].plot(modes, dec_times, marker="s", label="Decrypt")
    axs[0, 0].set_yscale("log")
    axs[0, 0].legend()

    axs[0, 1].plot(modes, psnr_values, marker="o", color="black")

    ratio = [dec_times[i] / enc_times[i] for i in range(len(modes))]
    axs[1, 0].plot(modes, ratio, marker="o", color="purple")

    max_psnr = max(psnr_values)
    diff = [max_psnr - v for v in psnr_values]
    axs[1, 1].plot(modes, diff, marker="o", color="green")

    plt.tight_layout(rect=[0, 0, 1, 0.96])
    plt.savefig(f"{GRAPH_DIR}/aes_comparison_figure.png", dpi=300)
    plt.close()

if __name__ == "__main__":
    main()
```

- create_bmp.py

```
from PIL import Image

img = Image.open("input2.jpg")


img = img.convert("RGB")


img.save("input2.bmp")
print("input2.bmp successfully created as RGB BMP.")
```